In this code sample, data is persisted in its basic, unenriched form. Our task then enriches the data and updates the database record. We could have used the simpler Task.start/1 function, but we used Task.Supervisor.start_child/2 because it ensures that tasks are given time to complete when an application shuts down. This is important to consider because applications stop and start during deployments.

This approach makes the basic data available immediately, and the enriched data will be available after the enrichment period. Our app would feel fast while making this API call, so the experience would feel significantly better to the end user.

This approach has a few major problems that are so severe that I've regretted it every time I've used it. Our app will spawn as many tasks as it can. If our API is hit with a large throughput, then the downstream requests will be significant. We've given up control over the parallelism of our async code, which is a big drawback! We also have no bookkeeping of the request. What if the API crashes, or our app reboots while processing? We have no guarantees that this code will ever finish!

With these drawbacks in mind, let's look at the last approach.

Use Oban

Oban is an Elixir job processing library that uses Postgres (or SQLite3) for coordinating and managing jobs. Oban is most similar to GoodJob[3] and Sidekiq.[4]

Oban uses processes to isolate jobs and provides complete control over job concurrency. Each job executes inside of its own process, which is started and handled by an Oban queue process.

Oban doesn't have the same problems that Task does. Oban controls the amount of concurrency automatically, has complete bookkeeping of our jobs, and handles application crashes well.

It's easy to define an Oban worker. Here's what our code might look like if we used Oban:

3. https://github.com/bensheldon/good_job
4. https://github.com/sidekiq/sidekiq